🎬 BUILD A MOVIE RECOMMENDATION SYSTEM

Google Colab → Machine Learning → Streamlit

PART A — PROJECT INTRODUCTION

SLIDE 1 — PROJECT CHALLENGE

🎬 **Build a Movie Recommendation System**

Client Requirement

“When a user selects a movie, the system should recommend other movies that are similar.”

Example:

User selects:

🎬 Inception

System recommends:

🎬 Interstellar
🎬 The Prestige
🎬 Tenet
🎬 Memento

Our Goal

Build a working movie recommendation product from scratch.

SLIDE 2 — WHAT ARE WE GOING TO BUILD?

Our project will have two major stages:

PART 1 — Core AI / ML

Movie Dataset

↓

Data Cleaning

↓

Feature Engineering

↓

TF-IDF

↓

Cosine Similarity

↓

Recommendation Function

PART 2 — Applied AI

Saved ML Components

↓

Python

↓

Streamlit

↓

🎬 Movie Recommendation Web App

SLIDE 3 — OUR DATASET

We will use the **Pablinho Movies Dataset** from Hugging Face.

Dataset:

Pablinho/movies-dataset

Source:

huggingface.co/datasets/Pablinho/movies-dataset

The dataset contains movie-related information such as:

Title
Overview
Genre
Popularity
Vote_Count
Vote_Average
Original_Language
Release_Date
Poster_Url

SLIDE 4 — WHY USE A REAL DATASET?

Instead of creating:

5 movies

10 movies

20 movies

we work with a much larger real-world movie dataset.

This allows us to experience:

Real Dataset

↓

Data Problems

↓

Data Processing

↓

Machine Learning

↓

Application

This is closer to an industry workflow.

SLIDE 5 — WHAT TYPE OF RECOMMENDER?

There are several recommendation approaches.

Collaborative Filtering

Uses:

Users

+

Ratings

+

Watch History

Content-Based Recommendation

Uses:

Movie Information

+

Movie Features

Today

We will build:

Content-Based Movie Recommendation

SLIDE 6 — HOW WILL OUR SYSTEM WORK?

Our system will ask:

"How similar is one movie to another?"

We will use:

Genre

+

Movie Overview

Example:

Inception

+

Science Fiction

+

Thriller

+

Dream-based story

Then find movies with similar textual content.

PART B — GOOGLE COLAB

SLIDE 7 — OPEN GOOGLE COLAB

Open:

Google Colab

Create:

New Notebook

Rename it:

Movie_Recommendation_System

SLIDE 8 — WHY GOOGLE COLAB?

Colab gives us:

- Python environment
- Easy package installation
- Interactive execution
- Easy experimentation
- No local Python setup required

Today:

Colab = Our ML Laboratory

SLIDE 9 — INSTALL REQUIRED LIBRARIES

Run:

!pip install datasets pandas scikit-learn joblib

Libraries

Library	Purpose
datasets	Load Hugging Face dataset
pandas	Data processing
scikit-learn	ML/text processing
joblib	Save ML objects

SLIDE 10 — IMPORT LIBRARIES

```
from datasets import load_dataset
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

import joblib
```

Cosine similarity is a metric used in Machine Learning (ML) to measure how similar two items are based on the direction they point, rather than their size. Imagine it as looking at two arrows drawn from the same starting point: it measures the angle between those arrows, completely ignoring how long or short they are.

PART C — LOAD HUGGING FACE DATASET

SLIDE 11 — LOAD THE DATASET

Run:

dataset = load_dataset("Pablinho/movies-dataset")

Then:

print(dataset)

Question

What did Hugging Face return?

https://huggingface.co/datasets/Pablinho/movies-dataset

SLIDE 12 — UNDERSTAND THE DATASET

Check the available split:

```
dataset
```

Then:

```
dataset["train"]
```

Convert it to Pandas:

```
df = dataset["train"].to_pandas()
```

SLIDE 13 — FIRST LOOK AT OUR DATA

Run:

df.head()

Now we can see our movie records.

Example:

- Title
- Overview
- Genre
- Popularity
- Vote_Average
- ...

SLIDE 14 — HOW MANY MOVIES?

Run:

```
df.shape
```

The output gives:
(number of rows, number of columns)

Think:

- How many movie records?
- How many features?

SLIDE 15 — CHECK COLUMN NAMES

Run:

```
df.columns
```

We are particularly interested in:

- Title
- Genre
- Overview
- Vote_Average
- Popularity
- Release_Date
- Poster_Url

SLIDE 16 — UNDERSTAND THE DATA

Run:

```
df.info()
```

Why?

We want to know:

- Data types
- Missing values
- Number of records
- Available columns

SLIDE 17 — LOOK AT IMPORTANT COLUMNS

Run:

```
df[ [ "Title", "Genre", "Overview", "Vote_Average" ]].head(10)
```

Student Task

Read 10 movie records.

Ask:

What information could help us decide whether two movies are similar?

PART D — DATA CLEANING

SLIDE 18 — CHECK MISSING VALUES

Run:

df.isnull().sum()

Question

Are there missing values?

Which columns contain them?

SLIDE 19 — WHY DO MISSING VALUES MATTER?

Suppose:

Genre = NaN

Overview = NaN

Then our recommendation system receives incomplete information.

Therefore:

Bad / Missing Data

↓

Poor Features

↓

Poor Recommendations

SLIDE 20 — HANDLE TEXT MISSING VALUES

Run:

```
df["Genre"] = df["Genre"].fillna("")
df["Overview"] = df["Overview"].fillna("")
```

Check:

```
df[
  ["Genre", "Overview"]
].isnull().sum()
```

SLIDE 21 — CHECK DUPLICATES

Run:

```
df.duplicated().sum()
```

We can also check duplicate movie titles:

```
df["Title"].duplicated().sum()
```

SLIDE 22 — REMOVE DUPLICATE TITLES

For our simple project:

```
df = df.drop_duplicates(    subset=["Title"])
```

Check:

```
df.shape
```

Now we have cleaner movie records.

PART E — FEATURE ENGINEERING

SLIDE 23 — WHAT FEATURES SHOULD WE USE?

Our dataset contains many columns.

But our first recommendation system will focus on:

Genre

+

Overview

Why?

Because these describe the **content of the movie**.

SLIDE 24 — CREATE A FEATURES COLUMN

Run:

```
df["features"] = (    df["Genre"] + " " +    df["Overview"])
```

When Warning comes:-

```
df.loc[:, "features"] = df["Genre"] + " " + df["Overview"]
```

Now:

```
df[    ["Title", "features"]].head()
```

SLIDE 25 — WHAT HAPPENED?

Before:

- Title
- Genre
- Overview

After:

- Title
- Features

Example:

Inception

Sci-Fi Thriller

A skilled thief enters dreams...

We created one text representation of each movie.

PART F — TEXT → NUMBERS

SLIDE 26 — THE MACHINE DOESN'T UNDERSTAND TEXT LIKE WE DO

Our computer sees:

"Science Fiction Thriller"

We need:

Text

↓

Numbers

Why?

Because our algorithm needs numerical representations to compare movies.

SLIDE 27 — INTRODUCE TF-IDF

TF-IDF

Term Frequency – Inverse Document Frequency

TF-IDF converts text into numerical features.

Movie Text

↓

TF-IDF

↓

Numerical Vector

SLIDE 28 — SIMPLE TF-IDF IDEA

Suppose we have:

Movie A:

space astronaut planet

Movie B:

space astronaut galaxy

Movie C:

fish ocean sea

Words that help distinguish documents receive useful weights.

Goal:

Represent each movie as a vector.

SLIDE 29 — CREATE TF-IDF VECTORIZER

vectorizer = TfidfVectorizer(stop_words="english")

stop_words="english"

Helps remove common English words that usually don't add much meaning.

Stop words are common, high-frequency words (such as articles, prepositions, and conjunctions) that carry very little unique semantic meaning and are routinely filtered out during text processing and search tasks

SLIDE 30 — TRANSFORM MOVIE TEXT

Run:

```
tfidf_matrix = vectorizer.fit_transform(    df["features"])
```

Now:

9,000+ Movies

↓

TF-IDF

↓

Movie Vectors

SLIDE 31 — CHECK THE RESULT

Run:

tfidf_matrix.shape

You will get:

(number_of_movies, number_of_features)

Meaning:

Each movie is represented by a numerical vector.

PART G — SIMILARITY

SLIDE 32 — NOW WE HAVE A NEW QUESTION

We have:

Inception → Vector A

Interstellar → Vector B

The Batman → Vector C

Finding Nemo → Vector D

Question:

How similar are these vectors?

SLIDE 33 — COSINE SIMILARITY

We will use:

Cosine Similarity

It measures similarity between vectors.

Conceptually:

Similarity ≈ 1

↓

Very similar

Similarity ≈ 0

↓

Less similar

SLIDE 34 — CALCULATE COSINE SIMILARITY

Run:

similarity = cosine_similarity(tfidf_matrix)

Now we have:

Movie × Movie

similarity scores.

SLIDE 35 — UNDERSTAND THE SIMILARITY MATRIX

Conceptually:

	Movie A	Movie B	Movie C
Movie A	1.00	0.82	0.10
Movie B	0.82	1.00	0.12
Movie C	0.10	0.12	1.00

Why is the diagonal 1?

A movie is perfectly similar to itself.

PART H — BUILD THE RECOMMENDER

SLIDE 36 — SELECT A MOVIE

Let's use:

movie_title = "Inception"

Find its index:

index = df[df["Title"] == movie_title].index[0]

SLIDE 37 — GET SIMILARITY SCORES

scores = list(enumerate(similarity[index]))

Each result contains:

(movie_index, similarity_score)

Example:

(0, 1.00)
(15, 0.84)
(27, 0.78)
(42, 0.72)

The Python enumerate() function is a built-in tool used to iterate over a collection (like a list, tuple, or string) while automatically keeping track of the index counter.

SLIDE 38 — SORT SIMILARITY SCORES

Run:

scores = sorted(scores, key=lambda x: x[1], reverse=True)

Now the highest similarity appears first.

1.00

0.84

0.78

0.72

...

SLIDE 39 — REMOVE THE MOVIE ITSELF

The first result is:

Inception → Inception

Similarity = 1.00

We don't want that.

So:

scores = scores[1:6]

Now we have the top 5 other movies.

SLIDE 40 — DISPLAY MOVIE TITLES

```
for movie_index, score in scores:
        print(      df.iloc[movie_index]["Title"]    )
```

Example:

Interstellar

The Prestige

Tenet

Memento

...

🎉 **Our recommendation algorithm works!**

SLIDE 41 — WHY CREATE A FUNCTION?

Instead of repeating our code every time:

recommend_movies("Inception")

recommend_movies("Interstellar")

recommend_movies("The Batman")

We create one reusable function.

SLIDE 42 — RECOMMENDATION FUNCTION

```
def recommend_movies(movie_title):
        if movie_title not in df["Title"].values:
        return []
        index = df[       df["Title"] == movie_title    ].index[0]
        scores = list(       enumerate(similarity[index])    )
        scores = sorted(       scores,       key=lambda x: x[1],       reverse=True    )
        scores = scores[1:6]
        recommendations = []
        for movie_index, score in scores:
                recommendations.append(          df.iloc[movie_index]["Title"]       )
          return recommendations
```

SLIDE 43 — TEST THE FUNCTION

Run:

recommend_movies("Inception")

Then:

recommend_movies("Interstellar")

Then:

recommend_movies("The Batman")

Student Challenge

Test at least **5 movies**.

SLIDE 44 — CHECK YOUR RESULTS

Create a table:

Discussion

Are all recommendations sensible?

Selected Movie	Recommendation 1	Recommendation 2	Recommendation 3
Inception			
Interstellar			
The Batman			
Avatar			
Titanic			

PART J — ADD MORE INFORMATION

SLIDE 45 — WE HAVE MORE THAN JUST TITLES

Our dataset contains:

- Title
- Genre
- Overview
- Vote_Average
- Popularity
- Release_Date
- Poster_Url

We can use these later to make our application more attractive.

SLIDE 46 — SHOW SIMILARITY SCORE

Modify our recommendation function to return:

Movie Name

+

Similarity Score

Instead of only:

```
recommendations.append(    df.iloc[movie_index]["Title"])
```

we can store:

```
recommendations.append({    "title": df.iloc[movie_index]["Title"],    "score": score})
```

SLIDE 47 — TEST THE SCORE

Example:

Inception

↓

Interstellar 0.82

The Prestige 0.76

Tenet 0.71

Memento 0.68

Remember:

Similarity score ≠ movie rating.

It measures how similar our algorithm considers the movie content.

PART K — CLIENT CHANGE REQUEST

SLIDE 48 — CTO CHANGE REQUEST

Client says:

“The recommendation algorithm works, but customers cannot use a Google Colab notebook.”

They want:

🎬 **A WEB APPLICATION**

SLIDE 49 — COLAB IS OUR LAB

Google Colab

Good for:

- Data exploration
- ML experiments
- Model development
- Testing

But the customer wants:

Browser

↓

Movie Application

↓

Recommendation

SLIDE 50 — WHAT NEEDS TO MOVE TO THE APPLICATION?

Our Streamlit application needs access to:

Movie Data

+

TF-IDF Vectorizer

+

Similarity Matrix

Therefore, we need to save our processed work.

PART L — SAVE OUR ML COMPONENTS

SLIDE 51 — SAVE THE TF-IDF VECTORIZER

Run:

joblib.dump(vectorizer, "vectorizer.pkl")

What is this?

The fitted TF-IDF vectorizer that learned our vocabulary and weights.

SLIDE 52 — SAVE THE SIMILARITY MATRIX

Run:

joblib.dump(similarity, "similarity.pkl")

Now our Streamlit app can load the precomputed similarity matrix.

SLIDE 53 — SAVE PROCESSED MOVIE DATA

Run:

```
df.to_csv(    "movies_processed.csv",    index=False)
```

Important

We are creating a processed application file.

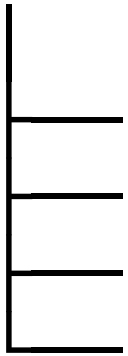
The original source remains:

- Hugging Face
- Pablinho/movies-dataset

SLIDE 54 — FILES WE NEED

Download these from Colab:

MovieRecommendation/



- app.py
- movies_processed.csv
- vectorizer.pkl
- similarity.pkl

app.py will be created in VS Code.

PART M — STREAMLIT

SLIDE 55 — OPEN VS CODE

Create:

MovieRecommendation

Open this folder in VS Code.

Place:

movies_processed.csv

vectorizer.pkl

similarity.pkl

inside the folder.

SLIDE 56 — INSTALL STREAMLIT

Open the VS Code terminal.

Run:

python -m pip install streamlit pandas scikit-learn joblib

If required:

py -m pip install streamlit pandas scikit-learn joblib

SLIDE 57 — CREATE app.py

Create:

app.py

First test:

```
import streamlit as st
st.title(    "🎬 Movie Recommendation System")
st.write(    "Welcome to our Movie AI application!")
```

SLIDE 58 — RUN STREAMLIT

In the terminal:

```
python -m streamlit run app.py
```

Open:

```
http://localhost:8501
```

Checkpoint

If you see:

🎬 Movie Recommendation System

your Streamlit setup works.

PART N — LOAD OUR ML SYSTEM

SLIDE 59 — IMPORT LIBRARIES

Update app.py:

```
import streamlit as st
import pandas as pd
import joblib
```

SLIDE 60 — LOAD MOVIE DATA

df = pd.read_csv("movies_processed.csv")

Check:

st.write(df.head())

Run the application.

The Streamlit app can now access our movie data.

SLIDE 61 — LOAD THE SIMILARITY MATRIX

Add:

similarity = joblib.load("similarity.pkl")

Now:

Streamlit

↓

Movie Data

+

Similarity Matrix

SLIDE 62 — LOAD TF-IDF VECTORIZER

Add:

vectorizer = joblib.load("vectorizer.pkl")

Our application now has the important ML components.

PART O — BUILD THE STREAMLIT UI

SLIDE 63 — CREATE THE TITLE

```
st.title(    "🎬 Movie Recommendation System")
st.write(    "Select a movie and discover similar movies.")
```

SLIDE 64 — MOVIE DROPDOWN

Create:

movie = st.selectbox("Select a movie", df["Title"].values)

The user sees:

Select a movie:

[Inception ▼]

SLIDE 65 — ADD THE BUTTON

```
if st.button(    "🎬 Recommend Movies"):
            st.write(        "Finding similar movies..."    )
```

Now our UI can respond to user actions.

PART P — CONNECT STREAMLIT TO OUR ML LOGIC

SLIDE 66 — FIND SELECTED MOVIE

Inside the button:

```
index = df[    df["Title"] == movie].index[0]
```

Now we know which movie the user selected.

SLIDE 67 — GET SIMILAR MOVIES

```
scores = list(    enumerate(similarity[index]))
```

Sort:

```
scores = sorted(    scores,    key=lambda x: x[1],    reverse=True)
```

Remove selected movie:

```
scores = scores[1:6]
```

SLIDE 68 — DISPLAY RECOMMENDATIONS

```
st.subheader(    "You may also like:")
for movie_index, score in scores:
        movie_name = df.iloc[       movie_index    ]["Title"]
        st.write(       "🎬",       movie_name    )
```

PART Q — ADD MOVIE INFORMATION

SLIDE 69 — SHOW GENRE

Inside the loop:

```
genre = df.iloc[    movie_index]["Genre"]
st.write(    "Genre:",    genre)
```

SLIDE 70 — SHOW RATING

```
rating = df.iloc[    movie_index]["Vote_Average"]
st.write(    f"⭐ Rating: {rating}")
```

SLIDE 71 — SHOW SIMILARITY

st.write(f"Similarity: {score:.2f}")

Example:

🎬 Interstellar

Genre: Science Fiction

⭐ Rating: 8.4

Similarity: 0.82

PART R — ADD POSTER

SLIDE 72 — USE POSTER_URL

Our dataset contains:

Poster_Url

Get it:

```
poster = df.iloc[    movie_index]["Poster_Url"]
```

SLIDE 73 — DISPLAY POSTER

st.image(poster, width=180)

Now the recommendation looks more like a real application.

PART S — IMPROVE THE USER EXPERIENCE

SLIDE 74 — RECOMMENDATION CARD

Each recommendation can show:

🎬 Interstellar

[POSTER]

Genre:
Science Fiction, Drama

⭐ Rating: 8.4

Similarity: 0.82

Goal:

Don't just show data.
Create a useful user experience.

SLIDE 75 — ADD MOVIE OVERVIEW

```
overview = df.iloc[    movie_index]["Overview"]
st.write(    overview)
```

Now users can understand **why the recommendation may be interesting**.

PART T — HANDLE ERRORS

SLIDE 76 — WHAT IF DATA IS MISSING?

Some movie records may have:

Missing Poster

Missing Overview

Missing Genre

Our application should not crash.

Good application:

Missing Data

↓

Handle Gracefully

↓

Continue Application

SLIDE 77 — CHECK POSTER

```
if poster:
        st.image(    poster,    width=180   )

Otherwise:
st.write(   "Poster not available")
```

SLIDE 81 — TEST CASES

Test	Expected
Select valid movie	Recommendations
Select another movie	Different recommendations
Poster exists	Poster displayed
Poster missing	App should continue
Missing genre	App should continue
Missing overview	App should continue